NeuroVision AI - Production Deployment Guide

Comprehensive guide for deploying NeuroVision AI across bare-metal servers, Docker containers, and cloud environments (Render, AWS EC2, DigitalOcean).

---

1. System Requirements

Hardware Prerequisites

- **Minimum:** 2 CPU cores, 4 GB RAM, 10 GB free disk space.
- **Recommended (Production):** 4+ CPU cores, 8 GB+ RAM, optional NVIDIA GPU with CUDA 12.x / Apple Silicon Metal MPS for ultra-fast training.

Software Prerequisites

- Python 3.10, 3.11, 3.12, or 3.13
- Docker Engine 24.0+ & Docker Compose v2 (optional for containerized deployment)
- Git & curl

---

2. Local Bare-Metal Deployment

Step 1: Clone the Codebase

```
git clone https://github.com/your-username/NeuroVision-AI.git
cd NeuroVision-AI
```

Step 2: Set up Virtual Environment

```
python3 -m venv venv
source venv/bin/activate  # On Windows: venv\Scripts\activate
```

Step 3: Install Production Dependencies

```
pip install --upgrade pip
pip install -r requirements.txt
```

Step 4: Configure Environment Variables

```
cp .env.example .env
# Edit .env to set your secure production keys
```

Step 5: Initialize SQLite Database & Pre-Seed Samples

```
python run.py init-db
python run.py seed
```

Step 6: Launch Production WSGI Server (Gunicorn)

```
# Production launch using 4 worker threads:
gunicorn --bind 0.0.0.0:5000 --workers 2 --threads 2 --timeout 120 app:app
```

Access the application at `http://localhost:5000`.

---

3. Containerized Deployment with Docker

Step 1: Build the Docker Image

```
docker build -t neurovision-ai:latest .
```

Step 2: Run the Docker Container

```
docker run -d \
  --name neurovision_app \
  -p 5000:5000 \
  -e FLASK_ENV=production \
  -e SECRET_KEY="your-production-secret-key" \
  -v $(pwd)/database:/app/database \
  -v $(pwd)/static/uploads:/app/static/uploads \
  -v $(pwd)/static/gradcam_outputs:/app/static/gradcam_outputs \
  -v $(pwd)/static/reports:/app/static/reports \
  --restart unless-stopped \
  neurovision-ai:latest
```

Step 3: Verify Container Health

```
docker ps
curl http://localhost:5000/api/health
```

Step 4: Launching with Docker Compose

```
docker compose up -d --build
```

To stop the services:

```
docker compose down
```

---

4. Cloud Deployment on Render

Render is an ideal zero-config PaaS platform for deploying the Flask + Gunicorn medical service.

Method A: Web Service Deployment (Dockerfile)

1. Fork or push this repository to your GitHub account.
2. Sign in to your Render Dashboard.
3. Click **New +** and select **Web Service**.
4. Connect your GitHub repository.
5. Configure the service settings:
   - **Name:** `neurovision-ai`
   - **Environment:** `Docker`
   - **Region:** Choose the region nearest to your users (e.g., Singapore, Frankfurt, Oregon).
   - **Branch:** `main`
   - **Plan:** Free or Starter (512MB to 2GB RAM).
6. Set Environment Variables:

- SECRET_KEY: <generate-a-random-32-byte-hex-string>
- FLASK_ENV: production
- PORT: 5000

7. Click **Create Web Service**. Render will automatically build the Docker image, run dependency installations, and expose your live HTTPS URL (e.g., https://neurovision-ai.onrender.com).

---

5. Cloud Deployment on AWS EC2 or DigitalOcean (Ubuntu 22.04 LTS)

Step 1: Server Provisioning & System Packages

```
sudo apt update && sudo apt upgrade -y
sudo apt install -y python3-pip python3-venv git nginx libgl1 libglib2.0-0
```

Step 2: Deploy Source Code

```
cd /var/www
sudo git clone https://github.com/your-username/NeuroVision-AI.git neurovision
cd neurovision
sudo python3 -m venv venv
sudo ./venv/bin/pip install -r requirements.txt
sudo ./venv/bin/python run.py seed
sudo chown -R www-data:www-data /var/www/neurovision
```

Step 3: Configure Systemd Service

Create /etc/systemd/system/neurovision.service:

```
[Unit]
Description=NeuroVision AI Gunicorn Medical Service
After=network.target

[Service]
User=www-data
Group=www-data
WorkingDirectory=/var/www/neurovision
Environment="PATH=/var/www/neurovision/venv/bin"
Environment="FLASK_ENV=production"
ExecStart=/var/www/neurovision/venv/bin/gunicorn --workers 3 --bind
127.0.0.1:5000 --timeout 120 app:app
Restart=always

[Install]
WantedBy=multi-user.target
```

Enable and start the service:

```
sudo systemctl daemon-reload
sudo systemctl enable neurovision
sudo systemctl start neurovision
```

Step 4: Configure Nginx Reverse Proxy

Create `/etc/nginx/sites-available/neurovision`:

```
server {
    listen 80;
    server_name your-medical-domain.com;

    client_max_body_size 20M;

    location / {
        proxy_pass http://127.0.0.1:5000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }

    location /static/ {
        alias /var/www/neurovision/static/;
        expires 30d;
    }
}
```

Enable site & obtain SSL Certificate:

```
sudo ln -s /etc/nginx/sites-available/neurovision /etc/nginx/sites-enabled/
sudo nginx -t
sudo systemctl restart nginx

# Install free Let's Encrypt SSL
sudo apt install -y certbot python3-certbot-nginx
sudo certbot --nginx -d your-medical-domain.com
```

---

6. Security & HIPAA/GDPR Compliance Best Practices

1. **Patient Data Residency:** The SQLite database is local to the server. For enterprise production, schedule periodic encrypted snapshots (`sqlite3 neurovision.db ".backup backup.db"`).
2. **Payload Size Guardrails:** The Flask app enforces a strict `16 MB` limit via `MAX_CONTENT_LENGTH` to protect against denial-of-service upload attacks.
3. **MIME-Type & Extension Sanitation:** Strict validation ensures only validated image streams (`PNG, JPG, JPEG, WEBP`) can pass to OpenCV decoding.
4. **HTTPS Encryption:** In production, always terminate traffic via HTTPS (TLS 1.3) to safeguard patient scans in transit.